CampusStay: design and implementation report

SEZG583 Assignment 1 | Option B | TEAM DETAILS PENDING

DRAFT: actual team details, contribution entries, reflections and submission links must be completed.

1. Problem, transaction and scope

A campus guesthouse has limited rooms of a given type on a given date. Visitors can compete for the final room; a client can repeat a booking after losing its response; a payment provider can decline or answer slowly. These cases require coordinated business behavior across independent services.

CampusStay books one room for one night. Inventory owns dated stock, Reservation owns the booking lifecycle, Payment owns mock charges/refunds and Notification owns mock delivery. The main transaction uses all four services through REST HTTP. Peer IDs travel in messages; there are no cross-database foreign keys.

Inventory and Reservation use Python/FastAPI. Payment and Notification use native PHP/PDO. The Windows target is XAMPP PHP/Apache with its MySQL-labelled MariaDB engine. A small optional PHP console demonstrates API health, booking state, replay and conflict. It has no database credentials.

The selected scope is mandatory Challenges 1 and 2 plus **Challenge 3C: saga with compensation**. Supplementary tests exercise timeout and stopped-service recovery. Authentication, real payment/email, a full customer UI and Assignment 2 Docker/Minikube deployment are outside this delivery.

Submission identity

Team: TEAM DETAILS PENDING. Actual member details and contributions belong in Appendix A. The final video must be the team's own 8-10 minute live demonstration with every member speaking. This report has 14 planned main pages, followed by appendices.

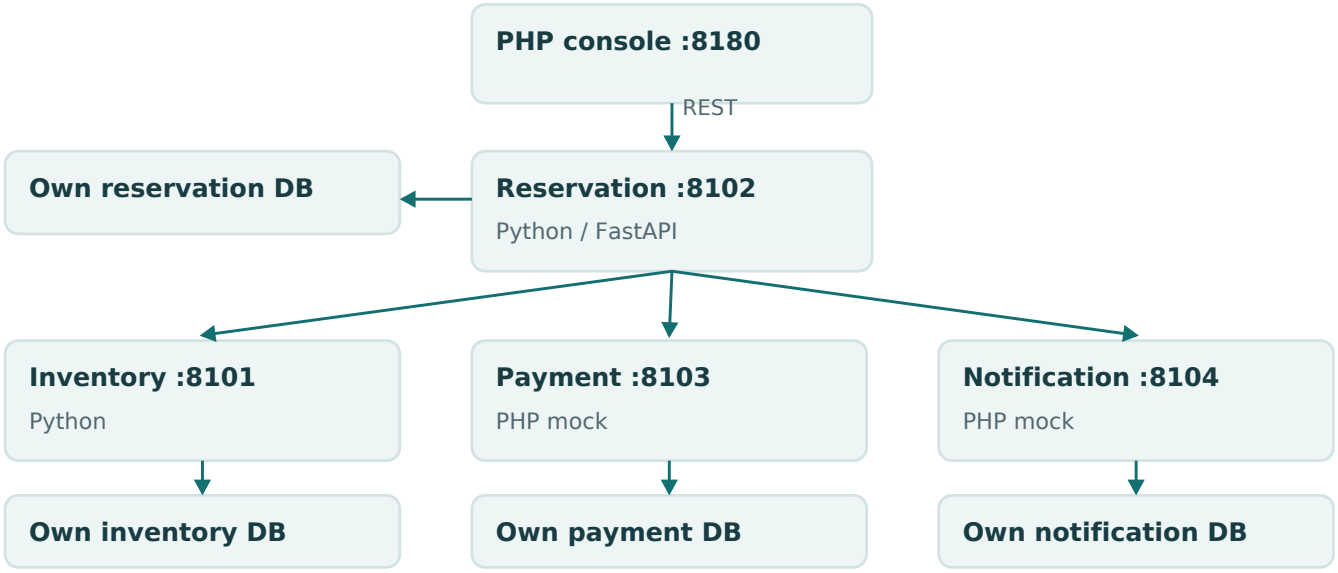
2. Business rules and traceability

Rule	Business decision	Implementation	Verification
BR1	Stay date cannot be in the past	Inventory/Reservation rules.py and booking claim	Date cases; past-date API rejection
BR2	Stock stays between zero and capacity	Inventory reserve; conditional UPDATE; SQL CHECK	Rule cases; 20 parallel requests
BR3	One key means one payload and immutable response	Local idempotency; Reservation claim/cache	Fingerprint cases; simultaneous replay
BR4	Confirmation requires payment	Reservation rules.py and saga.py	Transition cases; complete workflow
BR5	Release once; refund charged payment before release	Reservation saga; Inventory change_hold	Compensation cases; step-3 decline
BR6	Amount is a positive integer in minor units	Payment rules.php and SQL CHECK	Native PHP amount cases
BR7	Refund requires a charge; repeat gives one refund	Payment rules/repository; UNIQUE payment_id	PHP cases; concurrent refund check
BR8	Confirmation delivery requires confirmed state	Notification rules.php	Native PHP cases; retry workflow

Rules are service-local, rather than embedded in the UI. Database constraints add enforcement. Test names identify accepted and rejected decisions. There are 26 Python business cases, 14 native PHP business cases and nine other Python cases covering configuration/process ownership.

Money uses integer minor units: INR 2500 is 250000 paise. Inventory supplies the price; the client does not choose the charged amount. Supplied evidence uses synthetic guest and recipient values.

3. Decomposition, deployment and ownership



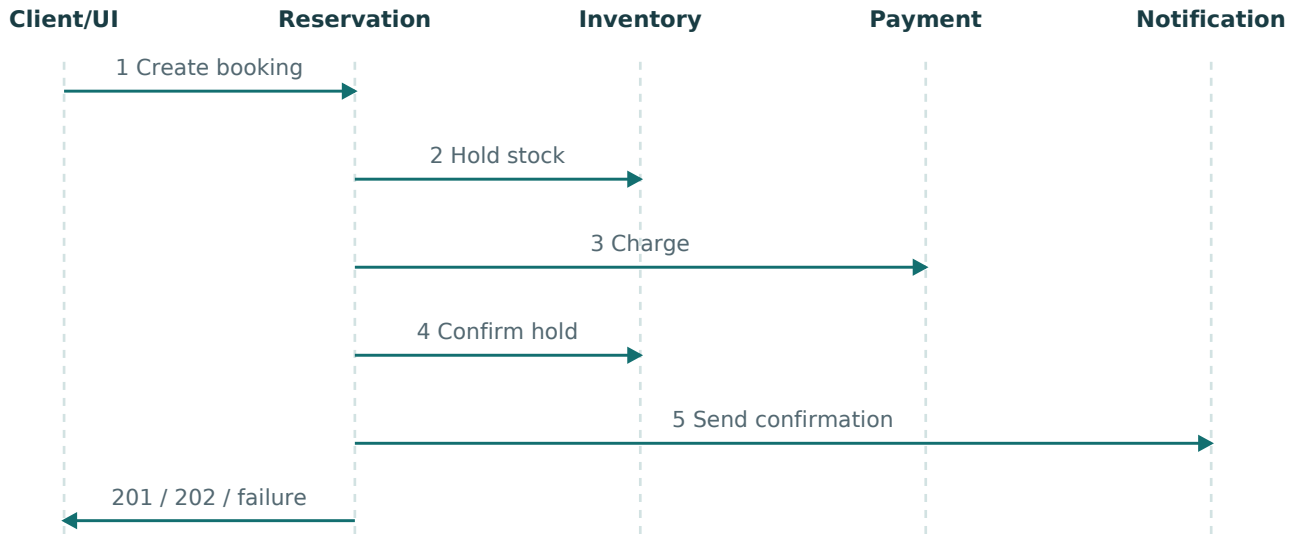
Service	API port	Database / restricted login
Inventory, Python	8101	campusstay_inventory / cs_inventory
Reservation, Python	8102	campusstay_reservation / cs_reservation
Payment mock, PHP	8103	campusstay_payment / cs_payment
Notification mock, PHP	8104	campusstay_notification / cs_notification

Each service folder contains its source, SQL, tests, dependency declaration, environment example and independent run.py. Python dependencies are locked. PHP uses built-in extensions; composer.json declares requirements. Infrastructure templates are vendored locally, avoiding runtime imports from a peer service.

Each application user has only SELECT/INSERT/UPDATE/DELETE on its own schema. Provisioning uses a separate administrator. The UI listens on 8180 and calls fixed API addresses through a PHP gateway. Diagnostic scripts separately open each service's own credentials; this is evidence tooling, not service integration.

Windows PHP launchers generate private Apache configurations using XAMPP, serving public/ only without editing the global configuration. Linux validation uses PHP's multi-worker development server. The combined launcher is optional: every business service also runs independently.

4. Interaction and booking lifecycle



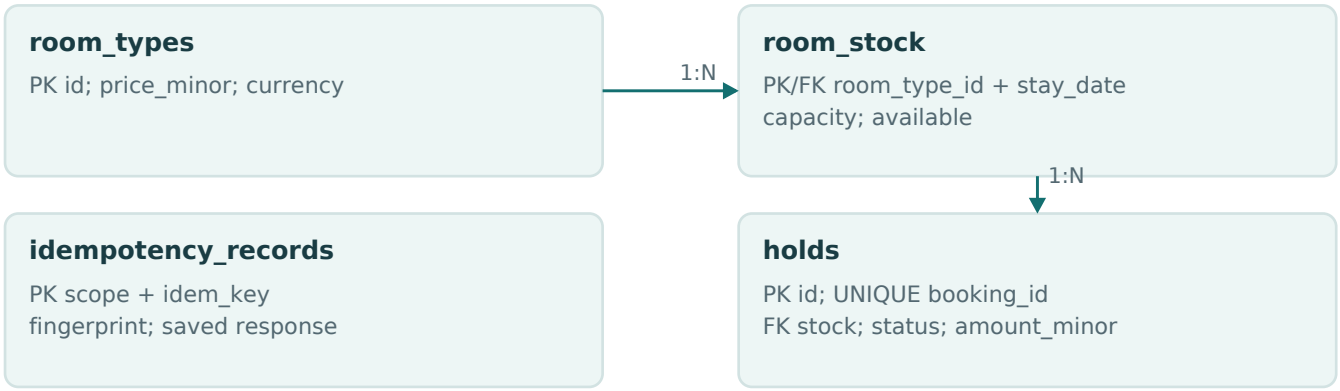
A client submits a booking with an Idempotency-Key. Reservation durably claims the key and booking before remote calls, then acquires a per-booking session lock and resumes persisted state. Stable downstream keys are derived from booking ID and operation.

Normal states are PENDING, RESERVED, PAYMENT_PENDING, PAID and CONFIRMED. Inventory confirms the hold after the charge. Notification failure leaves a CONFIRMED booking with notification_status PENDING; recovery retries delivery without charging again.

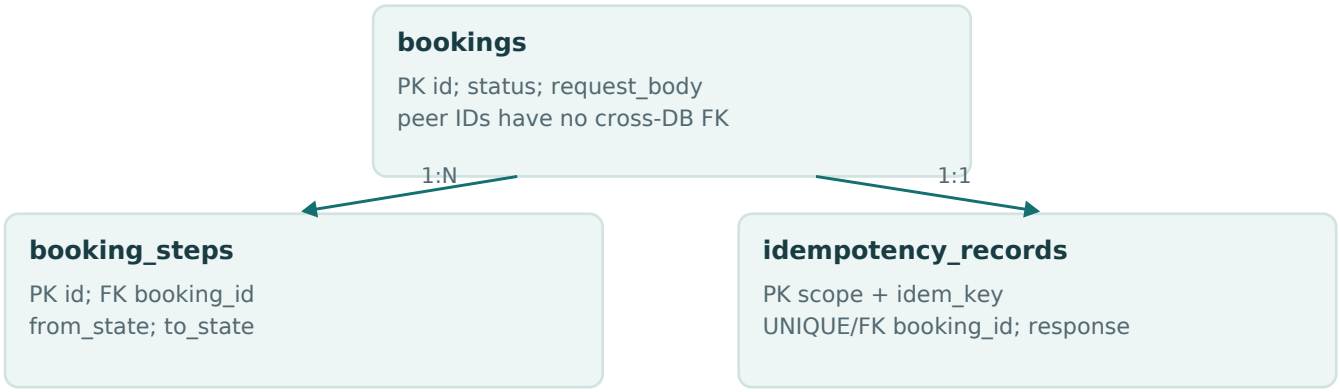
A declined payment passes through COMPENSATING to CANCELLED. Inventory release is idempotent. When compensation involves a charge, refund precedes release. An uncertain payment becomes PAYMENT_UNCERTAIN; a timeout alone does not release the room.

Transitions are stored in booking_steps. The saga session lock is on an autocommit connection: no SQL transaction remains open across network calls. Recovery uses the same state machine and business rules.

5. Data model: Inventory and Reservation



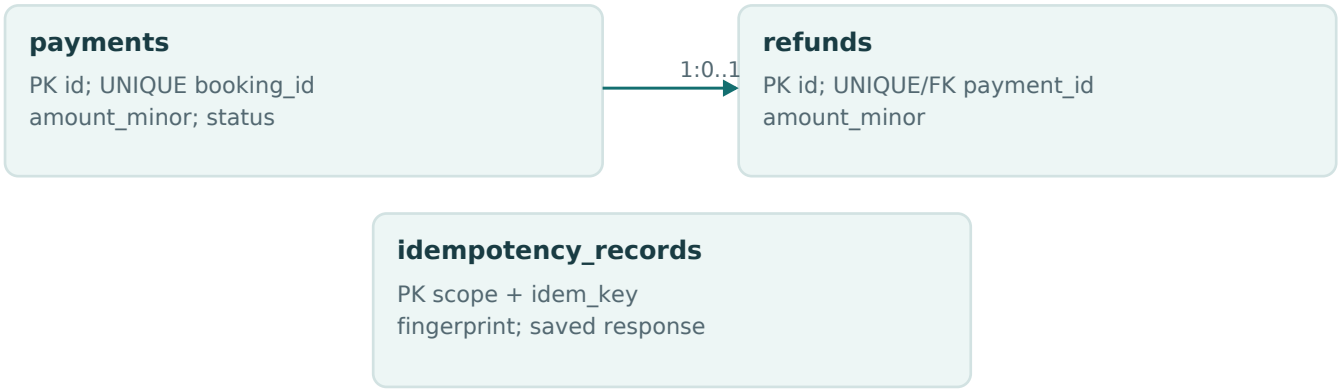
Inventory owns room_types, room_stock, holds and idempotency_records. Stock's primary key is (room_type_id, stay_date). Conditional decrement and CHECK constrain available stock. Each hold has a UNIQUE booking_id and a local foreign key to dated stock. Its price is an Inventory snapshot.



Reservation owns bookings, booking_steps and idempotency_records. A booking stores the validated request, state, observed payment ID, notification status and durable failure. The idempotency record has a unique booking_id. The booking is inserted before its local foreign-key record in one claim transaction. Peer room/payment IDs have no cross-service foreign keys.

UUIDs use CHAR(36), bodies use JSON, timestamps use UTC DATETIME(6), and connections use READ COMMITTED. Every schema uses InnoDB.

6. Data model: Payment and Notification



Payment owns payments, refunds and idempotency_records. UNIQUE payments.booking_id prevents repeated charges even with different keys. UNIQUE refunds.payment_id prevents multiple refunds and is a local foreign key. Payment modes are success, decline and slow_success. Only CHARGED payments become REFUNDED.



Notification owns notifications and idempotency_records. UNIQUE booking_id identifies the delivery record; status and attempts support fail_once and retry. Notification accepts confirmed state from its trusted internal caller and never reads Reservation's database. These unauthenticated demo APIs must stay local.

Each idempotency table has a (scope, idem_key) primary key, SHA-256 fingerprint and saved status/body. A local effect and its cached response commit together. Reservation additionally persists its booking claim before the distributed workflow and saves the first client response separately.

No service SQL joins another service's database. All 12 directed peer-database connections are tested and denied. There are 12 application tables across the four schemas.

7. Versioned API contract

All business routes below have prefix **/api/v1**. Exact schemas, headers, enums and responses are in /openapi.json and docs/openapi/*.json.

Service	Method and suffix	Purpose
Inventory	GET /room-types	List room types for console
Inventory	POST /room-types	Create dated priced stock
Inventory	GET /room-types/{id}/availability	Read availability
Inventory	POST /holds	Reserve one unit
Inventory	GET /holds/{booking_id}	Inspect hold
Inventory	POST /holds/{booking_id}/confirm	Confirm stock
Inventory	POST /holds/{booking_id}/release	Release once
Reservation	POST /bookings	Create/replay booking
Reservation	GET /bookings/{id}	Current booking state
Reservation	GET /bookings/{id}/steps	Transition audit
Reservation	POST /bookings/{id}/recover	Resume workflow
Payment	POST /payments	Mock charge
Payment	GET /payments/{booking_id}	Resolve outcome
Payment	POST /payments/{booking_id}/refund	Refund once
Notification	POST /notifications	Mock delivery
Notification	GET /notifications/{booking_id}	Delivery state
Notification	POST /notifications/{booking_id}/retry	Retry delivery

Each service also has GET /health and /openapi.json. Health checks its own DB. Python services offer interactive /docs. PHP contracts are JSON for an API client. The Postman collection is optional: supplied test clients do not require Postman.

8. Request, errors and observability

Example booking request: replace the room ID with the created stock ID and choose a non-past date.

```
{
  "room_type_id": "11111111-1111-4111-8111-111111111111",
  "stay_date": "2030-01-10",
  "guest_ref": "campus-visitor-01",
  "recipient": "visitor@example.test",
  "payment_mode": "success",
  "notification_mode": "success"
}
```

Send Content-Type: application/json, Idempotency-Key: a-new-request-key and X-Correlation-ID: a-demo-correlation. Same key and semantic payload replay the original status/body. Changed payload with that key returns 409. New business intent needs a new key.

```
{
  "error": {
    "code": "IDEMPOTENCY_CONFLICT",
    "message": "This key was already used with a different request payload.",
    "details": null
  }
}
```

Outcomes include 201 created, 200 current state, 202 unfinished, 402 mock decline, 404 missing, 409 conflict/no stock, 422 invalid input and 503 unavailable. A cached POST 202 stays 202 after recovery; GET shows current state.

Correlation IDs propagate downstream. Structured JSON logs identify service and request context. Evidence saves correlated logs per challenge. Secrets stay in ignored local environment files and are not logged in responses.

9. Challenge 1: contested capacity

The resource is one room_stock row for a room type/date. Checking availability in application memory cannot reserve the last unit: two callers can observe the same value. Inventory performs a conditional decrement and checks the affected-row count inside its local transaction.

```
UPDATE room_stock
SET available = available - 1
WHERE room_type_id = %s AND stay_date = %s
  AND available > 0;
```

The transaction creates the hold and saves its successful response. InnoDB row locking serializes updates to that row. A per-booking named lock and UNIQUE booking_id prevent repetition under different keys. CHECK constraints independently protect the stock range.

```
.\.venv\Scripts\python.exe scripts\challenge_concurrency.py `
  --requests 20 --capacity 5
```

Recorded result: 20 requests; capacity 5; HTTP counts {'409': 15, '201': 5}. The script creates isolated synthetic stock, releases 20 calls together, then checks responses and SQL state. Exactly N confirmed bookings, charges and holds must exist, with zero remaining stock and no negative value.

The evidence/challenge-1-concurrency folder contains results.json (responses/start times), database_state.json (service-local SQL snapshots), output.txt (assertions) and correlated *_logs.jsonl. This is a bounded correctness test, not a production throughput claim.

10. Challenge 2: durable idempotency

A key is scoped by operation. After validating the request, a service computes a canonical semantic fingerprint and acquires a namespace-scoped GET_LOCK. Under that lock it reads the durable record. Matching payload returns the saved status/body; a different fingerprint returns 409. A new local operation saves its effect and cached response in one InnoDB transaction, with a savepoint for business-error rollback.

READ COMMITTED lets a waiting lock owner see the preceding owner's committed record. GET_LOCK is session-scoped rather than released by COMMIT; connection closure releases it. Names are namespaced and hashed to fit the database limit.

Reservation claims the booking/key together. Simultaneous callers converge to one booking ID and one active saga owner. The first response is cached atomically; recovery does not replace it. UNIQUE business IDs remain necessary when a caller changes keys.

```
.\.venv\Scripts\python.exe scripts\challenge_idempotency.py
```

The script verifies three sequential replays, changed-payload rejection, 20 simultaneous first calls and one business effect in each participating service. It compares response bodies/statuses and SQL counts. Recorded result: 20 simultaneous calls; changed payload returned 409.

Evidence/challenge-2-idempotency contains results.json, database_state.json, output.txt and logs. Additional tests concurrently use different keys for the same booking and verify one hold, one charge and one refund, protecting business identity independently of request identity.

11. Challenge 3C: saga and compensation

Durable stages are (1) create booking, (2) hold stock and (3) charge payment, followed by stock confirmation and notification. The challenge declines Payment at step 3 after Inventory has committed the hold. Reservation cannot roll back another service's SQL transaction. It records COMPENSATING, calls release and records CANCELLED.

```
.\.venv\Scripts\python.exe scripts\challenge_saga.py
```

Recorded states are PENDING, RESERVED, PAYMENT_PENDING, COMPENSATING and CANCELLED. The original response is 402. SQL snapshots show a RELEASED hold, restored stock, declined payment and no charge or notification. Three repeated releases do not increase stock again. A later valid booking uses the restored room.

Evidence/challenge-3c-saga includes initial_check.json, database_after_compensation.json, database_after_repeated_compensation.json, results.json, output.txt and logs. Snapshots use each owner's credentials; services gain no peer SQL permissions for evidence collection.

Supplementary recovery

When Payment is slow or stopped, Reservation retains the hold and resolves the Payment API outcome before retrying with the same key. A timeout does not mean no charge. Stopped Notification leaves a paid booking confirmed; recovery retries delivery. The suite actually stops/restarts owned processes and checks immutable original POST responses after recovery.

This is a saga, not distributed ACID. A permanently unknown provider outcome requires reconciliation/manual intervention in production. The assessed extra remains 3C; no full completion of alternatives 3A-3D is claimed.

12. Verification and evidence provenance

Latest suite: **PASS**, finished 2026-10-18T15:47:07.353102+00:00. Actual runtime: **Linux; Python 3.12.14; PHP 8.3.6; database 10.11.14-MariaDB-0ubuntu0.24.04.1**. This is real native PHP, MariaDB/InnoDB and Python execution. It is not a native Windows run. Run Windows Apache startup and the full suite on the submission machine.

Check	Included verification
Business-rule units	26 Python + 14 native PHP cases
Configuration/process units	Nine Python cases
End-to-end	Success, decline, slow payment and notification retry
Concurrency	20 requests, five rooms, exactly five successes
Idempotency	Sequential and 20 simultaneous first calls; conflict
Saga	Step-3 decline, restored stock, repeat compensation
Data boundaries	All 12 peer accesses denied; wrong password rejected
Business identity	Different-key hold, charge and refund uniqueness
Service outage	Stop/restart actual Payment and Notification processes
Browser	PASS on Linux; 10 browser checks, separately timestamped

Evidence/tests/run_summary.json records versions, OS, timestamps, command exits and SHA-256 hashes for service/script/UI sources. Raw outputs and SQL snapshots support assertions. UI validation is separately dated. All data are synthetic; there is no real payment or email.

The guide includes fresh-clone rehearsal, prerequisites, configuration and SQL initialization. A packaging-time source-only Git clone check is recorded separately; it is not a fabricated team history. After substantive edits, rerun scripts/run_all.py and inspect the current outputs before recording.

13. Decisions and tradeoffs

Decision	Benefit	Cost or limitation
Separate schemas/users	Enforceable service ownership	Four connections and provisioning
Python and native PHP	HTTP language independence; XAMPP fit	Infrastructure compatibility to maintain
Atomic stock UPDATE	Safe last-unit arbitration	Popular stock row is contended
Named lock + durable cache	Concurrent first calls converge	Session cleanup/isolation need care
Persisted saga	Visible state and resumable compensation	Temporary inconsistency; no global rollback
Stable key + UNIQUE business ID	Safe lost-response retries	Conflicts must be explicitly modelled
Immutable POST + current GET	Replay preserves original result	Clients poll GET after 202
Local mock providers	Repeatable failure/timeout demonstrations	No real provider integration
Private Windows Apache	Concurrent PHP; owned public roots	Requires compatible XAMPP modules
API-only console	Accessible demonstration without DB coupling	Local use; no user authentication

All SQL transactions are local. No HTTP call occurs inside an open database transaction, limiting lock duration. Infrastructure is vendored; business state and responsibilities remain service-owned.

MariaDB GRANT treats unescaped underscores as wildcards. Setup escapes schema names and rejects same-named application accounts on other hosts to avoid ambiguous privilege matching. Runtime services never receive administrator credentials.

14. Reproduction, limits and submission

Follow BEGINNER_WINDOWS_GUIDE.md from software installation through GitHub, SQL, UI, tests, fresh clone, personal fields, PDF regeneration and video upload. Windows Python 3.12 wheels are bundled. PDF regeneration has separate documentation dependencies; generated PDFs are supplied.

Run configure.py, init_databases.py and doctor.py --require-stopped --apache-check before scripts/run.py. Stop the normal launcher before run_all.py, which owns its own processes. Each service can also launch independently. No batch files are required.

Limits: one room/one night; trusted local callers; mock providers; no production authentication, broker or operations platform. Permanent payment uncertainty can retain stock until reconciliation. Future work can add authentication, provider reconciliation, an outbox/queue, metrics and Assignment 2 containers while retaining separate databases.

GitHub: PENDING - actual faculty-accessible repository

Supporting materials: PENDING - actual supporting-material folder

Video: PENDING - actual team-recorded video

References and acknowledgement

The controlling brief is Assignment1_Microservices_Design_and_Development.docx, SEZG583 Assignment 1. The baseline adapts the earlier AI-assisted PostgreSQL prototype; substantial assistance is declared in Appendix C. Students must verify actual course requirements.

- XAMPP FAQ: https://www.apachefriends.org/faq_windows.html
- MariaDB GET_LOCK: https://mariadb.com/docs/server/reference/sql-functions/secondary-functions/miscellaneous-functions/get_lock
- MariaDB GRANT: https://mariadb.com/docs/server/reference/sql-statements/account-management-sql-statements/grant
- PHP server: https://www.php.net/manual/en/features.commandline.webserver.php

Appendix A. Actual contributions

PENDING: enter real names, IDs, responsibilities, actual work and genuine commit references.

Importing the AI-assisted baseline is not a claim of independent authorship. Record work actually performed and link genuine commits or reviewed evidence. Do not invent past commits or use another person's Git identity.

Use scripts/fill_submission_details.py to enter actual details and URLs. After substantive work and a native Windows run, rebuild this report and check its main-page limit.

Appendix B. Personal reflections

PENDING: each actual member must complete their own half-page reflection.

Each member must write approximately half a page in their own words: actual work, a difficult problem personally addressed, tests performed, learning, future improvements and assistance used. Supplied worksheets are prompts, not completed reflections.

Appendix C. AI-use declaration

This initial implementation and documentation are substantially AI-assisted. ChatGPT/Codex helped interpret the brief and previous prototype; migrate PostgreSQL behavior to MySQL/MariaDB; implement the Python/PHP split, SQL, browser console and setup; generate/review tests; debug actual execution; and prepare instructional materials.

Assistance covered business logic, concurrency/idempotency/saga infrastructure, code review and documentation. Files are a reviewable baseline, not proof of independent student authorship. Included evidence is from real automated runs with actual platform/timestamps recorded. It does not claim native Windows validation or a student-recorded video.

Students must disclose the actual assistance used in their own work, review and understand the code, test it, make genuine contributions, retain verification notes if requested and write reflections themselves. Update AI_USE_DECLARATION.md if subsequent assistance changes this statement.

The team must supply its real repository, contribution entries, IDs, reflections and video. No plagiarism-detection score, originality certificate or institutional acceptance is guaranteed. Distinguish the assisted baseline from work students actually performed.